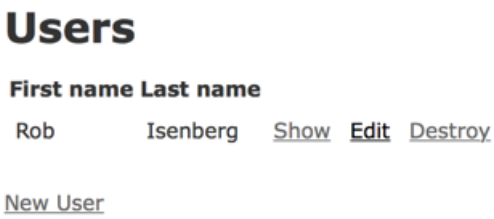
Let's prove that our data is now persisted even if we delete the database container. First, we need to store some data: add one or more users through the Rails scaffold. In the following figure, I've created a single user for myself.



Now that we have some data stored, let's stop the database container:

```
$ docker-compose stop database
Stopping myapp_database_1 ... done
```

and then delete it (you will need to confirm when asked):

```
$ docker-compose rm database
Going to remove myapp_database_1
Are you sure? [yN] y
Removing myapp_database_1 ... done
```

Next, we recreate it and start it up with:

```
$ docker-compose up -d database
```

If this works, we should see our user data, exactly as it was before we deleted the database container. Reload the browser (at http://localhost:3000/users), and…our data is still there. Hurrah!

But Where Actually Is My Data?



We said that Docker manages an area of the filesystem for named volumes, but where actually is it? We can find out where our db_data named volume (which Compose prefixes with our app folder to become myapp_db_data) is located by running:

```
$ docker volume inspect --format '{{ .Mountpoint }}' myapp_db_data
/var/lib/docker/volumes/myapp_db_data/_data
```

As we can see, named volumes are stored in /var/lib/docker/volumes/.[8] On Linux, this will be a path on the local filesystem, but on macOS or Windows, this refers to the path *inside* the Dockerhost VM.

8. https://docs.docker.com/storage/#choose-the-right-type-of-mount